

```
In [2]: import pandas as pd
import numpy as np

import warnings
warnings.filterwarnings(action = 'ignore')

from IPython.display import Image

for i in [pd, np]:
    print(i.__name__, i.__version__)
```

pandas 2.1.4

numpy 1.26.4

```
In [3]: Image('5.png')
```

Out[3]: 1-1 데이터의 분류

데이터의 종류에는 어떤 것들이 있는지 정리해봅니다.

0. 데이터셋 소개

Titanic

[Titanic](#) 탑승객의 생존 유무를 담은 데이터셋 입니다.

Data	Description	Dictionary
PassengerId	Passenger Id, Index	
Survived	Survival	0 = No, 1 = Yes
Pclass	Ticket class	1 = 1st, 2 = 2nd, 3 = 3rd
Name	Name	
Sex	Sex	
Age	Age in years	
Sibsp	# of siblings / spouses aboard the Titanic	
Parch	# of parents / children aboard the Titanic	
Ticket	Ticket number	
Fare	Passenger fare	
Cabin	Cabin number	
Embarked	Port of Embarkation	C = Cherbourg, Q = Queenstown, S = Southampton

다양한 형식의 데이터를 가지고 있으며, 여러 가지 아이디어를 생각하고 시도할 만한 요소가 많게끔 기획된 데이터셋입니다.

```
In [5]: df_titanic = pd.read_csv('data/titanic.csv')
df_titanic.head()
```

```
Out[5]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futrelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

1.데이터의 종류

정형 데이터(Structured Data)

- 구조화된 형식을 가지며, 행과 열로 이루어진 표 형태를 이룹니다.

열(Column)은 특정 유형의 데이터를 포함하고, 행(Row)은 고유한 레코드를 나타냅니다.

Ex) CSV 파일. 관계형 DB의 테이블, ...

정형 데이터의 분류

1. 질적 데이터(Qualitative data): 품질, 특성 또는 특성의 유무를 나타내는

비수치적 데이터입니다. 범주형 데이터라고도 불립니다. 수학적 계산이 가능하더라도 의미는 없습니다.

- 명목형 데이터(nominal data):

순서의 개념이 없습니다. Ex) 색상, 국가

- 서열형 데이터(ordinal data):

순서의 개념이 있습니다. Ex) 직급, 난이도

2. 양적 데이터(Quantitative data): 양이나 수치를 나타낼 수 있는 데이터로,
수학적 계산이 가능한 데이터입니다.

- 연속형 데이터(continuous data):

실수와 같은 연속적인 수치를 나타내는 데이터입니다. Ex) 키, 무게, 온도

- 이산형 자료(discrete data):

정수와 같이 불연속적인 수치를 나타내는 데이터입니다. Ex) 출석 인원수, 무사고 일수

[Ex.1]

df_titanic의 질적 데이터들을 살펴봅니다.

In [6]:

```
'''
코드는 DataFrame에서 특정 열들의
고유한 값들을 추출하는 과정을 나타냅니다.

1. `df_titanic[['Survived', 'Pclass', 'Sex', 'Embarked']]`:
   이 부분은 DataFrame에서 'Survived', 'Pclass', 'Sex', 'Embarked'
   열들을 선택하는 것을 의미합니다.
   이렇게 선택된 열들은 새로운 DataFrame으로 반환됩니다.

2. `.apply(lambda x: x.loc[x.notna()].unique().tolist())`:
   이 부분은 선택된 각 열에 대해 특정 함수를 적용하는 것을 나타냅니다.
   여기서는 Lambda 함수를 사용하여 각 열에 대해 고유한 값을 추출합니다.
'''
```

```
- `lambda x: x.loc[x.notna()]`:
    이 부분은 각 열에 대해 결측치를 제외한 값을 선택하는
    Lambda 함수를 정의합니다.
    `x.notna()`는 해당 열이 결측치가 아닌지를 확인하고,
    `x.loc[]`를 사용하여 결측치가 아닌 값을 선택합니다.
- `.unique().tolist()`:
    선택된 열의 고유한 값을 추출하여 리스트로 변환합니다.
    `.unique()` 함수는 고유한 값을 선택하고,
    `.tolist()` 함수는 해당 값을 리스트로 변환합니다.
```

이를 통해 각 열에 대해 결측치를 제외하고 고유한 값들을 추출하여
리스트 형태로 반환합니다. 결과적으로 각 열의 고유한 값들을
리스트로 나열한 형태의 **Series**가 반환됩니다.

```
'''

# 질적 데이터의 수준들을 가져옵니다.
# 각 열에 대한 연산을 수행하게 합니다.
df_titanic[['Survived', 'Pclass', 'Sex', 'Embarked']].apply(
# 결측을 제거한 후 유일값을 뽑습니다.
    lambda x: x.loc[x.notna()].unique().tolist()
)
```

```
Out[6]: Survived      [0, 1]
Pclass      [3, 1, 2]
Sex      [male, female]
Embarked    [S, C, Q]
dtype: object
```

Survived, Sex, Embarked는 명목형 데이터이고, Pclass는 서열형 데이터입니다.

```
In [16]: # PassengerId는 정수형이지만 명목형 데이터입니다. 사칙연산이 성립하지 않습니다.
# (PassengerId 1 + PassengerId 2 ≠ PassengerId 3)
df_titanic['PassengerId']
```

```
Out[16]: 0      1
         1      2
         2      3
         3      4
         4      5
```

```
...
886    887
887    888
888    889
889    890
890    891
```

Name: PassengerId, Length: 891, dtype: int64

Cabin은 명목형 데이터와 정수형 데이터가 결합되어 있고

단일 객체를 나타내는 것이 아닌 복수의 객체를 나타내도록 되어 있습니다.

C85, B53 B55, ...

[Ex.2]

df_titanic의 양적 데이터를 살펴봅니다.

```
In [5]: # 수치를 나타내는 정수형 데이터입니다.
df_titanic[['Age', 'SibSp', 'Parch']]
```

```
Out[5]:
```

	Age	SibSp	Parch
0	22.0	1	0
1	38.0	1	0
2	26.0	0	0
3	35.0	1	0
4	35.0	0	0
...	...	...	...
886	27.0	0	0
887	19.0	0	0
888	NaN	1	2
889	26.0	0	0
890	32.0	0	0

891 rows × 3 columns

```
In [6]: # 수치를 나타내는 실수형 데이터입니다.
df_titanic['Fare']
```

```
Out[6]:
```

0	7.2500
1	71.2833
2	7.9250
3	53.1000
4	8.0500
...	...
886	13.0000
887	30.0000
888	23.4500
889	30.0000
890	7.7500

Name: Fare, Length: 891, dtype: float64

[Ex.3]

Cabin의 첫자리는 객실 유형을 그 다음 숫자는 객실의 위치를 나타냅니다.

결측이 아닌 Cabin만을 뽑고, 객실 유형을 분리하여 Cabin_Type 변수를 만들고,

객실 위치를 분리하여 Cabin_No 변수를 만듭니다.

```
In [17]: df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].unique()
```

```
Out[17]: array(['C85', 'C123', 'E46', 'G6', 'C103', 'D56', 'A6', 'C23 C25 C27',
               'B78', 'D33', 'B30', 'C52', 'B28', 'C83', 'F33', 'F G73', 'E31',
               'A5', 'D10 D12', 'D26', 'C110', 'B58 B60', 'E101', 'F E69', 'D47',
               'B86', 'F2', 'C2', 'E33', 'B19', 'A7', 'C49', 'F4', 'A32', 'B4',
               'B80', 'A31', 'D36', 'D15', 'C93', 'C78', 'D35', 'C87', 'B77',
               'E67', 'B94', 'C125', 'C99', 'C118', 'D7', 'A19', 'B49', 'D',
               'C22 C26', 'C106', 'C65', 'E36', 'C54', 'B57 B59 B63 B66', 'C7',
               'E34', 'C32', 'B18', 'C124', 'C91', 'E40', 'T', 'C128', 'D37',
               'B35', 'E50', 'C82', 'B96 B98', 'E10', 'E44', 'A34', 'C104',
               'C111', 'C92', 'E38', 'D21', 'E12', 'E63', 'A14', 'B37', 'C30',
               'D20', 'B79', 'E25', 'D46', 'B73', 'C95', 'B38', 'B39', 'B22',
               'C86', 'C70', 'A16', 'C101', 'C68', 'A10', 'E68', 'B41', 'A20',
               'D19', 'D50', 'D9', 'A23', 'B50', 'A26', 'D48', 'E58', 'C126',
               'B71', 'B51 B53 B55', 'D49', 'B5', 'B20', 'F G63', 'C62 C64',
               'E24', 'C90', 'C45', 'E8', 'B101', 'D45', 'C46', 'D30', 'E121',
               'D11', 'E77', 'F38', 'B3', 'D6', 'B82 B84', 'D17', 'A36', 'B102',
               'B69', 'E49', 'C47', 'D28', 'E17', 'A24', 'C50', 'B42', 'C148'],
              dtype=object)
```

```
In [9]: import pandas as pd
```

```
data = {'A': [1, 2, 3], 'B': [[4, 5], [6], [7, 8, 9]]}
df = pd.DataFrame(data)
df
```



```
Out[9]:
```

	A	B
0	1	[4, 5]
1	2	[6]
2	3	[7, 8, 9]

```
In [10]: result = df.explode('B')
result
```

```
Out[10]:
```

	A	B
0	1	4
0	1	5
1	2	6
2	3	7
2	3	8
2	3	9

```
In [8]: '''
코드는 'Cabin' 열의 값이 결측치가 아닌 행들을 선택하고,
각 행의 'Cabin' 값을 공백을 기준으로 분할(split)하여
새로운 DataFrame을 생성하는 과정을 나타냅니다.

1. `df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin']`:
   이 부분은 'Cabin' 열의 값이 결측치가 아닌 행들을 선택하는 것을 의미합니다.
   `df_titanic['Cabin'].notna()`는 'Cabin' 열의 값이 결측치가 아닌지를
   나타내는 불리언 시리즈를 반환하고,
   이를 사용하여 결측치가 아닌 행들을 선택합니다.
   선택된 행들의 'Cabin' 열의 값들로 구성된 새로운 시리즈가 반환됩니다.
2. `.str.split(' ')`: 선택된 'Cabin' 열의 값들을 공백을 기준으로
   분할(split)합니다. 각 값은 공백을 기준으로 분할되어 리스트로 반환됩니다.
3. `.explode()`: 분할된 리스트를 행 단위로 '펼칩니다'.
   즉, 각 리스트의 원소들을 새로운 행으로 만들어 새로운 DataFrame을 생성합니다.
'''
```

```
4. `.apply(lambda x: pd.Series((x[:1], x[1:]), index=['Cabin_Type', 'Cabin_No']))`:
    각 행의 값을 이용하여 새로운 DataFrame을 생성하는 함수를 적용합니다.
    여기서는 Lambda 함수를 사용하여 각 행의 값을
    'Cabin_Type'과 'Cabin_No'로 분리하여 새로운 DataFrame을 생성합니다.
- `lambda x: pd.Series((x[:1], x[1:]), index=['Cabin_Type', 'Cabin_No'])`:
    이 부분은 Lambda 함수를 정의하여 각 행의 값을
    'Cabin_Type'과 'Cabin_No'로 분리하는 역할을 합니다.
    `x[:1]`은 각 행의 리스트의 첫 번째 원소를 선택하여
    'Cabin_Type'에 할당하고, `x[1:]`은 나머지 원소들을 선택하여
    'Cabin_No'에 할당합니다. 이렇게 생성된 값들을 이용하여
    새로운 Series를 생성합니다.
결과적으로 위의 코드는 'Cabin' 열의 값을 공백을 기준으로 분할하여
'Cabin_Type'과 'Cabin_No'로 나누고, 이를 새로운 DataFrame으로 생성합니다.
이러한 처리 과정을 통해 'Cabin' 열의 정보를
보다 세부적으로 분리하고 정리할 수 있습니다.
'''
```

```
# 여러 객실을 포함한 경우를 처리하기 위해,
# 1. 미결측치를 가져온후 Cabin 문자열을 공백으로 나눕니다.
#   → 리스트로 구성된 Series가 나옵니다.
# 2. 리스트의 요소 각각을 Series의 요소로 꺼내어,
#   Series에 하나의 Cabin 정보가 위치하도록 합니다.
# 3. Cabin의 맨앞자리는 객실 유형을 나타냅니다.
#   이를 Cabin_Type으로 이후는 객실 번호를 나타내는 Cabin_No로 분리합니다.
df_titanic.loc[df_titanic['Cabin'].notna(), 'Cabin'].str.split(' ')\
    .explode()\
    .apply(
        lambda x: pd.Series((x[:1], x[1:]), index=['Cabin_Type', 'Cabin_No'])
    )
```

Out[8]:

	Cabin_Type	Cabin_No
1	C	85
3	C	123
6	E	46
10	G	6
11	C	103
...	...	...
872	B	53
872	B	55
879	C	50
887	B	42
889	C	148

238 rows × 2 columns

비정형 데이터(Unstructured Data)

- 구조가 없거나 제한적인 구조를 가지고 있지 않은 데이터입니다. Ex) 텍스트, 음성, 비디오, ...

In [11]: `Image('6.png')`

Out[11]: **반정형 데이터(Semi-Structured Data)**

- 일정한 형식을 지니고 있지만 완전한 정형 데이터는 아닌 데이터입니다.

XML

```
<person>
  <name>John Doe</name>
  <age>30</age>
  <address>
    <city>New York</city>
    <zipcode>10001</zipcode>
  </address>
  <contacts>
    <email>john.doe@example.com</email>
    <phone>123-456-7890</phone>
  </contacts>
</person>
```

JSON

```
{
  "person": {
    "name": "John Doe",
    "age": 30,
    "address": {
      "city": "New York",
      "zipcode": "10001"
    },
    "contacts": {
      "email": "john.doe@example.com",
      "phone": "123-456-7890"
    }
  }
}
```

```
}  
}  
}
```

2. 정형 데이터의 구성 요소

Series

- 1차원의 **인덱스**가 부여된 자료 저장 공간
 - **인덱스(Index)**: 요소에 부여된 색인 데이터

```
In [9]: # Series를 만들어봅니다.  
s = pd.Series(  
    [1, 5, 3, 10, 7], index=['a', 'b', 'c', 'd', 'e']  
)  
s
```

```
Out[9]: a      1  
       b      5  
       c      3  
       d     10  
       e      7  
dtype: int64
```

```
In [10]: # 인덱스로 조회를 해봅니다.  
# 'a', 'd'의 값을 가져옵니다.  
s.loc[['a', 'd']]
```

```
Out[10]: a      1  
        d     10  
dtype: int64
```

```
In [11]: # 위치 인덱스로 탐색을 해봅니다  
# 처음과 마지막을 가져옵니다.  
s.iloc[[0, -1]]
```

```
Out[11]: a    1
         e    7
         dtype: int64
```

DataFrame

- 2차원의 **인덱스**가 부여된 자료 저장 공간.

- 인덱스(Index): 행(row)에 부여된 색인 데이터
- 컬럼(Column): 열(column)에 부여된 색인 데이터

```
In [12]: # df_titanic DataFrame의 상단 부분을 출력합니다.
         df_titanic.head()
```

```
Out[12]:
```

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
0	1	0	3	Braund, Mr. Owen Harris	male	22.0	1	0	A/5 21171	7.2500	NaN	S
1	2	1	1	Cumings, Mrs. John Bradley (Florence Briggs Th...	female	38.0	1	0	PC 17599	71.2833	C85	C
2	3	1	3	Heikkinen, Miss. Laina	female	26.0	0	0	STON/O2. 3101282	7.9250	NaN	S
3	4	1	1	Futelle, Mrs. Jacques Heath (Lily May Peel)	female	35.0	1	0	113803	53.1000	C123	S
4	5	0	3	Allen, Mr. William Henry	male	35.0	0	0	373450	8.0500	NaN	S

보통 여러 개의 Series가 모여 DataFrame을 구성합니다.

```
In [13]: # DataFrame의 하나의 컬럼은 Series입니다.
         df_titanic['PassengerId']
```

```
Out[13]: 0      1
         1      2
         2      3
         3      4
         4      5
         ...
        886    887
        887    888
        888    889
        889    890
        890    891
Name: PassengerId, Length: 891, dtype: int64
```

```
In [14]: # DataFrame의 하나의 행 또한 Series입니다.
```

```
df_titanic.iloc[0]
```

```
Out[14]: PassengerId      1
Survived                0
Pclass                  3
Name      Braund, Mr. Owen Harris
Sex                male
Age                22
SibSp                1
Parch                0
Ticket      A/5 21171
Fare                7.25
Cabin                NaN
Embarked            S
Name: 0, dtype: object
```

데이터 분석 실력 및 Level 3에 가장 중요한 포인트

- 데이터(프레임) 처리 능력

3. 빅데이터

3V (Volume, Velocity, Variety)

1. Volume(양) : 데이터의 규모나 양을 나타냅니다.
2. Velocity(속도): 데이터가 생성, 수집, 그리고 분석되는 속도를 나타냅니다.
3. Variety(다양성): 데이터 형식의 다양함을 나타냅니다.

5V (3V + Veracity, Value)

4. Veracity(정확성) : 데이터의 정확성과 신뢰성을 나타냅니다.
5. Value(가치): 데이터를 통해 얻을 수 있는 가치를 나타냅니다.

4. 모델

- **모델**(모형)의 사전적 의미: 대상을 근사화한 객체
- 통계 / 머신 러닝 관점에서의 의미

데이터의 근사화 → 데이터에 드러난 패턴/지식을 근사화한 객체

In []: